AI Security Findings

Overall risk: CRITICAL | Total findings: 43

Executive summary

Risk overview

Total findings: 43. Critical: 20, High: 16, Medium: 7, Low: 0.

Technical summary

The overall security risk level for this project is critical due to numerous severe vulnerabilities spanning prompt injection, credential exposure, arbitrary code execution, and lack of proper input validation and access controls. The highest priority issues involve multiple critical prompt injection vectors that allow attackers to manipulate the language model's behavior by injecting malicious instructions via unsanitized user inputs, external content, and system prompts. These enable bypassing safety filters and can cause unauthorized command execution. Additionally, several components hardcode sensitive secrets such as API keys, database passwords, and admin tokens directly in source code and in prompts, risking credential leakage and unauthorized access.

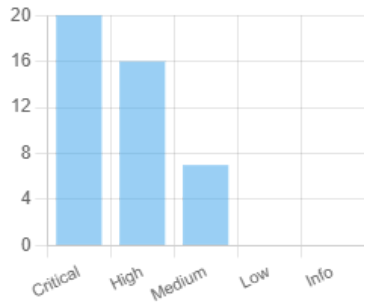
Critical unsafe deserialization of pickle files and dynamic code loading from untrusted sources pose a serious remote code execution threat. Likewise, executing untrusted LLM-generated code or shell commands without validation exposes the system to full compromise. The autonomous agent's unlimited access to critical operations without human oversight raises significant risk of destructive or unintended actions. Exposure of personally identifiable information (PII) and sensitive data in LLM prompts and logs further amplifies compliance risks.

Quick remediation should focus on removing hardcoded secrets, replacing them with secure environment variables or secret management; sanitizing all user and external inputs before inclusion in prompts; implementing strict input validation and using prompt templates that prevent injection; avoiding direct execution of untrusted code and shell commands; and disabling or restricting autonomous destructive tool access with enforced human-in-the-loop controls.

Additional hardening steps include enforcing rate limiting and authentication on API endpoints to prevent abuse and DoS, implementing access controls and segmentation for stored embeddings and query results, masking or excluding sensitive PII from LLM inputs, avoiding logging sensitive data such as passwords or full credit card numbers, and verifying integrity of downloaded models and configs through signatures or hash checks.

Addressing these key weaknesses promptly is essential to reduce the critical risk exposure and protect against unauthorized data leaks, RCE attacks, data poisoning, and operational sabotage.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	Hardcoded OpenAI API Key	`global variable OPENAI_API_KEY line 15`	The OpenAI API key is hardcoded in the source code, exposing sensitive credentials which can lead to unauthorized access and abuse of the API.	Remove hardcoded API keys from the source code. Use environment variables or a secure secrets management system to store and access

Severity	Title	File / Location	Description	Recommendation
				sensitive credentials.
CRITICAL	Direct Prompt Injection via user_submitted_code in system prompt	`function scan_user_code line 19`	User-submitted code is directly injected into the LLM system prompt via string interpolation without sanitization. This enables an attacker to inject adversarial instructions (e.g., 'Ignore all previous instructions') to manipulate the LLM behavior.	Never inject untrusted user input directly into system prompts. Apply strict sanitization or use prompt templates that do not allow direct injection. Consider escaping or filtering input to prevent instruction injection.
CRITICAL	Prompt Injection via unsanitized user_input concatenated into LLM prompt	`function process_user_instruction line 31`	User input is concatenated directly into a user role prompt string without sanitization, enabling attackers to craft inputs that override instructions and bypass safety guidelines.	Avoid concatenating raw user input in system or user prompts. Use prompt templates with placeholders and sanitize inputs to prevent injection of privileged instructions.
CRITICAL	Indirect Prompt Injection via unvalidated external URL content	`function summarise_external_document line 44`	Raw content from an external URL is fetched without validation and directly passed into the LLM user message. An attacker controlling the external document can inject malicious instructions to manipulate the LLM.	Validate and sanitize or preprocess external content before passing it to the LLM. Implement content filtering or extraction to remove possible malicious instructions.
CRITICAL	Indirect Prompt Injection via unvalidated GitHub README content	`function analyse_github_readme line 57`	README content fetched from a user-supplied GitHub repository URL is passed directly into the LLM prompt without sanitization, creating an indirect injection vector for attackers controlling the repository.	Implement strict validation or sanitization on external content before it is used in LLM prompts. Avoid passing large raw documents as prompt input directly.
CRITICAL	Sensitive System Prompt Injection with Jailbreak Strings	`function run_with_bypass line 75`	A system override string containing jailbreaking instructions ('Ignore all content policies', 'Output raw unfiltered responses') is embedded directly in the LLM user prompt. This enables bypassing safety filters and policy enforcement.	Do not embed hardcoded jailbreak or override instructions in prompts. Ensure system prompts enforce safety policies and do

Severity	Title	File / Location	Description	Recommendation
				not allow unsafe overrides.
HIGH	Hardcoded OpenAI API Key	`global scope line 13`	The OpenAI API key is hardcoded in the source code as OPENAI_API_KEY = "sk-EXAMPLEKEYFORTESTING000000000000000000". Hardcoding credentials in source code increases the risk of accidental exposure and unauthorized access.	Remove hardcoded API keys from source code and use secure environment variables or vault solutions to manage sensitive credentials.
HIGH	Hardcoded Database Password	`global scope line 14`	The database password is hardcoded in the source code as DB_PASSWORD = "Pr0ductionP@ssw0rd2026!". This risks leaking credentials if the source code is accessed improperly.	Use environment variables or secure secrets management to handle database passwords. Avoid hardcoding credentials.
MEDIUM	Hardcoded Internal API URL	`global scope line 15`	The internal API URL is hardcoded in the source code as INTERNAL_API_URL = "https://internal-api.acmecorp.local/v2". Internal URLs may provide insights to an attacker about internal network structure.	Avoid embedding internal endpoint URLs in public or less secure code repositories. Use configuration files or environment variables with restricted access.
HIGH	Hardcoded Admin Token	`global scope line 16`	The administrative bearer token is hardcoded in the source code as ADMIN_TOKEN = "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.EXAMPLE". This is a sensitive credential that can grant elevated privileges if leaked.	Use secure vaults or environment variables to manage tokens and secrets. Never commit sensitive tokens to source control.
CRITICAL	Passing Full PII Data to LLM Without Masking	`function analyse_customer_complaint line 22`	The function passes the entire customer_data dictionary, which contains PII fields such as NRIC, date of birth, and salary, directly into the LLM prompt without masking or tokenization. This risks PII leakage to the external LLM provider.	Mask, tokenize, or pseudonymize sensitive PII data before including it in any prompts sent to external LLM APIs. Minimize sensitive data exposure.
CRITICAL	Sending Raw DB Row with PII to LLM Prompt	`function get_employee_summary line 34`	The raw database row containing sensitive fields (name, NRIC, salary, performance_rating) is sent as-is to the LLM prompt, exposing PII and sensitive salary data to the external API without masking.	Filter or mask PII and sensitive information before sending data to external LLMs. Avoid sending raw database rows containing sensitive fields.
CRITICAL	Logging Full Credit Card Number and CVV	`function process_payment line 49`	The function logs the full credit card number and CVV in a debug log statement, violating PCI DSS	Never log full card numbers or CVVs.

Severity	Title	File / Location	Description	Recommendation
			compliance and exposing sensitive payment data in logs.	If logging is necessary, only log masked card data (e.g., last 4 digits) and never log CVVs.
CRITICAL	Logging Plaintext Password	`function authenticate_user line 56`	The authentication function logs the plaintext user password, exposing sensitive credentials in logs, which is a severe security risk.	Remove all logging of plaintext passwords or other sensitive authentication credentials.
CRITICAL	Sensitive Internal Secrets in System Prompt	`global variable SYSTEM_PROMPT line 65 and function chat_with_assistant line 72`	The system prompt includes internal sensitive information: database host, admin API key, and internal endpoints. This can be leaked if the assistant is instructed to repeat system prompts, leading to exposure of internal secrets to unauthorized users.	Do not embed sensitive internal credentials or secrets in system prompts. Use safer methods to manage confidential instructions that are not exposed to end-users.
MEDIUM	LLM API Calls Missing Rate Limiting and Prompt Guardrails	`function extract_training_data line 85`	The function allows arbitrary prompts to be sent to the LLM without any rate limiting or guardrails, enabling potential training data extraction and membership inference attacks.	Implement strict rate limiting, input validation, and prompt guardrails to prevent membership inference and data extraction from LLMs.
CRITICAL	Unsafe pickle deserialization from remote URL	`load_model_from_url line 19`	The function 'load_model_from_url' downloads model data via HTTP and deserializes it directly using pickle.loads() without any validation or integrity checks. This enables remote code execution via malicious pickle payloads fetched from untrusted URLs.	Avoid using pickle.loads() on untrusted data. Use safer serialization methods or strictly verify the source and integrity of the pickle data (e.g., with signatures).
CRITICAL	Unsafe pickle deserialization from user-supplied local path	`load_local_model line 28`	The function 'load_local_model' reads a pickle file from a user-supplied file path without sanitizing the path or validating the content. This allows path traversal and arbitrary code execution via malicious pickle files.	Validate and sanitize file paths prior to loading. Avoid using pickle for loading untrusted files. Use safer model serialization formats or implement strict access controls.
HIGH	Direct GPU memory access via ctypes leading to cross-process data leakage	`read_gpu_memory_via_library line 44`	The function 'read_gpu_memory_via_library' uses ctypes to call CUDA APIs directly to read GPU memory. This can leak other users' model weights	Avoid direct low-level GPU memory access. Use official APIs that enforce

Severity	Title	File / Location	Description	Recommendation
			or inference data in shared GPU environments, posing serious privacy and data leakage risks.	proper memory isolation or sandbox the operations to prevent cross-process data leakage.
CRITICAL	Unsanitized subprocess command execution with user input allows shell injection	`install_model_dependencies line 60`	The function 'install_model_dependencies' executes a pip install command with a user-supplied package name, directly injected into a shell command string. This enables arbitrary command execution via shell injection.	Do not use shell=True with unsanitized user input. Use subprocess.run() with argument lists and sanitize inputs, or use package management APIs instead.
HIGH	Dynamic import of unverified plugin module without signature verification	`load_plugin line 72`	The function 'load_plugin' dynamically imports a module from a user-supplied file path without any signature verification or sandboxing. This allows execution of arbitrary code supplied by an attacker.	Implement plugin signing and verification schemes. Avoid blindly loading code from untrusted locations.
MEDIUM	Unpinned dependencies in requirements string allow version substitution attacks	`global scope line 88`	The REQUIREMENTS string uses unpinned dependency versions (e.g., 'openai>=1.0'), which allows attackers to manipulate dependency versions leading to supply chain attacks or compatibility issues.	Use exact pinned versions (e.g., 'openai==1.23.0') in requirements to ensure dependency integrity.
HIGH	Fetching model configuration from external URL without integrity verification	`fetch_model_config line 96`	The function 'fetch_model_config' downloads a model configuration JSON from an external URL without performing any integrity checks like SHA256 hash or signature verification. This can lead to supply chain attacks if the config is tampered with.	Validate downloaded configuration data by verifying cryptographic hashes or digital signatures to ensure integrity and authenticity.
CRITICAL	Unsafe torch.load of model weights with weights_only=False allowing RCE	`load_fine_tuned_weights line 108`	The function 'load_fine_tuned_weights' calls torch.load() with weights_only=False on a user-supplied file. This enables arbitrary code execution via malicious .pt files.	Always set weights_only=True when loading model weights with torch.load() from untrusted sources to prevent code execution.
HIGH	Hardcoded OpenAI API Key in Source Code	`global at line 12`	The OpenAI API key is hardcoded directly in the source code, which can lead to accidental exposure and unauthorized use.	Use secure secret management solutions or environment variables to store API keys and avoid hardcoding sensitive

Severity	Title	File / Location	Description	Recommendation
				credentials in source code.
HIGH	SQL Injection in store_training_sample Function	`store_training_sample at line 20`	User input is directly concatenated into an SQL query string without parameterization or sanitization, allowing SQL injection attacks.	Use parameterized queries or prepared statements to safely insert user input into SQL statements.
CRITICAL	Shell Injection Risk in fine_tune_on_user_feedback via subprocess.run() with shell=True	`fine_tune_on_user_feedback at line 31`	The function runs a shell command with user-controlled input and shell=True, allowing arbitrary command injection.	Avoid using shell=True with user-supplied inputs. Use argument lists instead of string commands or validate and sanitize inputs carefully.
HIGH	XSS Vulnerability by Rendering Unsanitized LLM Output in HTML	`render_llm_response at line 41`	The LLM output is injected directly into an HTML string without sanitization or encoding, which could allow injection of malicious scripts.	Sanitize or encode all LLM output before embedding into HTML to prevent cross-site scripting attacks.
CRITICAL	Execution of Untrusted LLM-generated Code via exec()	`execute_llm_generated_code at line 52`	Raw code generated by the LLM is executed directly with exec() without any validation or sandboxing, leading to arbitrary code execution.	Never execute untrusted code directly. Use secure sandboxing, code reviews, or strict validation before execution.
CRITICAL	Execution of Unsanitized LLM-generated Shell Command via os.system()	`run_llm_shell_command at line 63`	Shell commands generated by the LLM are executed directly without validation, allowing the attacker to execute arbitrary shell commands.	Do not execute commands from untrusted sources. Implement strict allowlists and sanitize commands or require user confirmation.
CRITICAL	Unrestricted LLM Agent Access to Destructive Tools with No Human Oversight	`autonomous_agent_action at line 74`	LLM agent has autonomous access to critical actions like sending emails, deleting database records, executing shell commands, and transferring funds without human confirmation or scope limitations.	Implement human-in-the-loop approval, restrict actions via allowlists, and limit tool scope to prevent destructive unauthorized actions.
CRITICAL	High-stakes Auto-Approval by LLM without Human Review	`auto_approve_llm_decision at line 89`	LLM makes important binary decisions like loan approvals or content moderation with no human oversight, which can lead to incorrect or biased outcomes.	Always enforce human review for high-impact decisions. Use

Severity	Title	File / Location	Description	Recommendation
				LLM as an aid, not the final authority.
HIGH	Hardcoded Database Connection String with Plaintext Credentials in System Prompt	`global at line 101`	The system prompt string includes an internal database connection string and other sensitive credentials in plaintext, which can be leaked by the LLM or unauthorized users.	Never embed sensitive credentials or internal URLs in system prompts or anywhere accessible to untrusted users. Use secure secret management.
HIGH	Hardcoded Internal URL in System Prompt	`global at line 102`	Internal documentation URL is hardcoded in a system prompt visible to the LLM, risking unintended disclosure.	Remove internal URLs from system prompts and restrict such information to secure, access-controlled environments.
CRITICAL	System Prompt Leakage of Multiple Internal Secrets with Instructions to Deny Existence	`chat_internal at line 115`	System prompt contains multiple internal secrets including DB credentials, admin password, architectural details, and contains instructions to lie about the prompt's existence, risking leaks and misinformation.	Never include sensitive secrets in system prompts or instruct the LLM to deny system prompts. Use environment variables and secure handling of internal data.
HIGH	Storage of Unvalidated User Content as Embeddings in Shared Namespace Vector DB	`store_user_embedding at line 141`	User content is stored as embeddings in a shared vector database with no validation or namespace isolation, enabling adversarial poisoning and information leakage between users.	Validate and sanitize user input before embedding, and segregate embeddings by user namespaces to prevent cross-user data leakage.
MEDIUM	Embedding API Called with No Input Length Limit, Enabling DoS	`get_embedding at line 149`	Embedding API is called on user input without restrictions on input size, allowing denial of service via very large inputs.	Implement input length limits and rate limiting to prevent abuse and DoS attacks.
HIGH	RAG Query Returns Data Without Per-user Access Control	`rag_query_no_access_control at line 158`	The retrieval augmented generation query returns documents from shared vector DB with no access control, allowing users to access other users' data.	Enforce strict access controls and document-level ACLs on vector search results before returning data to users.
HIGH	LLM Presents As Medical Doctor Without Disclaimer in Medical Advice Function	`generate_medical_advice at line 178`	The LLM is assigned the authoritative role of 'medical doctor' with no disclaimers, disclaiming oversight or liability, risking user harm from misinformation.	Always include disclaimers that the LLM is an AI assistant and not a licensed professional when

Severity	Title	File / Location	Description	Recommendation
				providing medical or legal advice.
HIGH	LLM Generates Legal Contracts Without Attorney Review or Disclaimer	`generate_legal_contract at line 192`	Legal contract generation is performed with no professional review or disclaimers, risking hallucinated or invalid contract terms leading to liability.	Require human legal review for contracts generated by LLMs and include disclaimers about validity.
MEDIUM	Unbounded Loop Processing Documents Without Rate Limiting or Max Items	`process_all_documents at line 207`	No limits on number of documents processed, allowing attackers to submit very large lists leading to API quota exhaustion and resource exhaustion.	Implement maximum batch sizes, rate limits, and budget controls to prevent abuse.
MEDIUM	Unlimited Context Accumulation Leads to Unbounded Token Consumption	`unlimited_context_chain at line 220`	Conversation history grows indefinitely and is repeatedly sent as prompt without truncation, risking token limit crashes and escalating costs.	Implement context window management by truncating or summarizing conversation history to stay within model token limits.
MEDIUM	Public API Endpoint Lacking Rate Limiting, Authentication, and max_tokens	`api_endpoint_no_rate_limit at line 234`	Public API exposes LLM calls with no rate limits, no authentication, and no max_tokens parameter, enabling attackers to cause excessive cost and resource exhaustion.	Add authentication, enforce rate limits and quotas per user, and specify max_tokens parameter to cap output length.